

Before starting, make sure you **read all the questions** and understand what is asked of you. Write your answers to the questions in a separate sheet of paper and make sure you **justify yes/no answers**.

1. (18 points) Translate the following C snippet into LLVM intermediate representation and into MIPS.

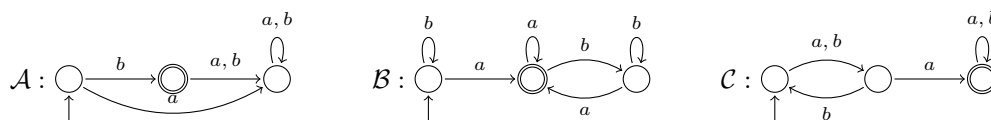
```
int gcd(int a, int b) {
    if (b == 0) return a;
    return gcd(b, a % b);
}
```

Tip: in LLVM, the modulus operator can be implemented using the `srem` instruction; in MIPS you use the “Hi” result of the division instruction.

2. (12 points) Split the following words into tokens (that is, lexeme-word pairs)

- (a) (4 points) `aa`
- (b) (4 points) `aab`
- (c) (4 points) `abbbab`

using the following automata with **decreasing priority from left to right**. **Tip:** remember to follow the “maximal munch” principle.



3. (26 points) Let $T = \{\text{id}, +\}$ be a set of terminals. Is the following grammar LL(2)? Does it define an LL(3) language?

(1)	S	$\rightarrow$	Exp
(2)	Exp	$\rightarrow$	Exp + Exp
(3)		$\rightarrow$	id
(4)		$\rightarrow$	id++

4. (10 points) Let $T = \{\text{cst}, *\}$ be a set of terminals and S be the initial non-terminal symbol. In the following **attribute grammar** the assignments on the right of the grammar rules should be understood as *semantic rules* a.k.a. *attribute functions*.

(1)	S	$\rightarrow$	FT	$\{a(T) := v(F); v(S) := v(T)\}$
(2)	T_0	$\rightarrow$	$*FT_1$	$\{a(T_1) := a(T_0) \times v(F); v(T_0) := v(T_1)\}$
(3)	T	$\rightarrow$	$*F$	$\{a(F) := a(T); v(T) := a(F) \times v(F)\}$
(4)	F	$\rightarrow$	cst	$\{v(F) := \text{lexval}(\text{cst})\}$

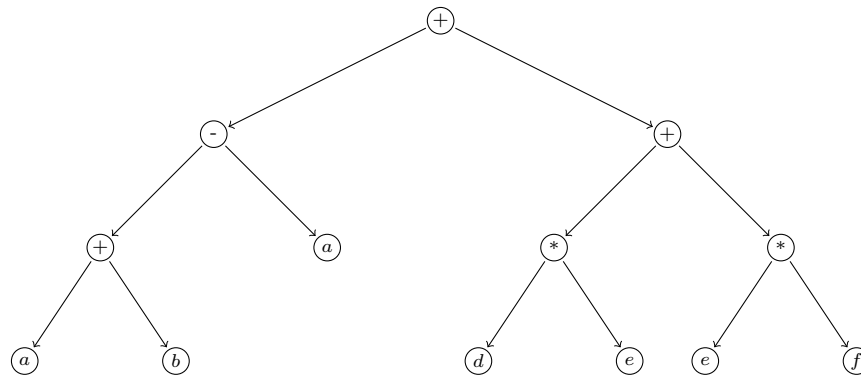
Tip: `lexval(cst)` refers to the numerical value of `cst` and so it is **not** an attribute.

- (a) (5 points) Classify the attributes, per non-terminal variable, into synthesized and inherited.
 - (b) (5 points) What is the *meaning* (i.e. the semantics) of a word derived from S ?
5. (10 points) During the course we have focused on conservative approximations of both type checking and reachability analysis. The exact versions of both tasks are in fact undecidable. Give an example of such an algorithm seen in class and a program which show that **we do not solve an undecidable problem**.
6. (20 points) Let $T = \{\text{cst}, \text{id}, +\}$ be a set of terminals. Is the following grammar SLR(1)? Is it LALR(1)?

(1)	S	$\rightarrow$	Exp
(2)	Exp	$\rightarrow$	Exp + Exp
(3)		$\rightarrow$	cst
(4)		$\rightarrow$	id

Hint: There are no Accept/Reduce or Accept/Shift conflicts, that is to say Accept can co-exist with at most one other action in the action-table cells.

7. (4 points) What is the minimal number of registers required to evaluate the expression tree shown below without using algebraic properties, nor additional memory?



8. (5 points) **Bonus:** In the above question, can you do better if you use algebraic identities? Why or why not?